# ATA 017 — Autenticação, perfis de acesso e trilha de auditoria

> **Data:** 02/10/2026
> **Projeto:** Plataforma local de gestão jurídica e automação assistida
> **Participantes:** Contratante/advogado; estagiário de Engenharia de Software (IA assistida)
> **Referência:** ATA 016 e `docs/atores-e-usuarios.md`
> **Carga horária da semana (28/09 a 02/10):** 30 horas

## Contexto

Com os dados persistidos (ATA 016), o acesso precisava ser restrito, conforme o requisito "o sistema terá login e área restrita" (ATA 002). Esta semana implementa autenticação local, perfis e registro de eventos relevantes.

## Decisões

1. **Senhas** guardadas apenas como hash PBKDF2-HMAC-SHA256 (240 mil iterações, sal aleatório por senha), usando somente a biblioteca padrão do Python. Política mínima: 10 caracteres, com letras e números.
2. **Sessão por token opaco** gerado aleatoriamente, válido por 8 horas; no banco fica apenas o hash do token.
3. **Bloqueio** da conta por 15 minutos após 5 senhas erradas seguidas. A mensagem de erro é sempre genérica ("Credenciais inválidas"), para não revelar se um e-mail existe.
4. **Perfis**: `advogado` e `estagiario` têm acesso interno; `cliente` existe no modelo, mas ainda não acessa nada (área do cliente é etapa futura).
5. **Trilha de auditoria** (tabela `auditoria`) protegida por gatilhos que impedem alteração e exclusão. Os registros contêm ação, usuário, entidade e data — **nunca** conteúdo de minuta, senha ou dado pessoal.
6. Nenhuma senha é escrita no código: usuários são criados por `scripts/criar_usuario.py`, que lê a senha de forma oculta.

## Entrega realizada

- `services/auth.py` (hash, login, sessão, logout, bloqueio e verificação de perfil).
- Esquema v2 em `services/database.py` (colunas de bloqueio, tabelas `sessao` e `auditoria`).
- `scripts/criar_usuario.py` e `scripts/popular_dados_ficticios.py` (banco de desenvolvimento com senhas aleatórias exibidas uma única vez).
- `tests/test_auth.py`.

## Validação técnica

Total acumulado: 19 testes, todos aprovados. Os novos 10 cobrem: hash com sal diferente a cada chamada, hash malformado, política de senha, login válido (e-mail sem diferenciar maiúsculas), token nunca guardado em texto puro, mensagem genérica, bloqueio mesmo com a senha correta depois das falhas, sessão expirada, logout, usuário inativo, auditoria sem senha e negação de acesso ao perfil `cliente`.

## Anexo técnico — trecho comentado

```
if not verificar_senha(senha, usuario["senha_hash"]):
    falhas = usuario["tentativas_falhas"] + 1
    bloqueio = agora(BLOQUEIO) if falhas >= MAX_FALHAS else None   # 5ª falha bloqueia
    conexao.execute("UPDATE usuario SET tentativas_falhas = ?, bloqueado_ate = ? WHERE id =
?", ...)
    raise ErroAutenticacao("Credenciais inválidas.")               # mensagem igual para
qualquer falha
```

`hmac.compare_digest` é usado na comparação dos hashes para evitar ataques por medição de tempo.

## Limitações e riscos

- O serviço ainda não usa HTTPS; aceita conexões somente de `127.0.0.1`. Se o sistema for exposto na rede do escritório, HTTPS passa a ser obrigatório.
- Não há redefinição de senha nem autenticação em dois fatores (fora do MVP).
- Risco: contas compartilhadas. Orientação: um usuário por pessoa, para a auditoria fazer sentido.

## Pendências para o contratante

- Confirmar quem terá perfil `advogado` (inicialmente Mauro e Patricia, conforme ATA 001).

## Próximo passo

Implementar o fluxo de revisão e aprovação da minuta pelo advogado.

---

Documento de acompanhamento do projeto. Não substitui documentos oficiais da UTFPR nem instrumentos jurídicos do escritório.